

# claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf\_c81dbe7a-7ff\agent-acb49306619d8d0bf.jsonl`
- SHA-256: `0fd63f98b6a51a74ed70891dae39714e90ce8e6e13e4fb6c9bdccea6eefd4a72`
- Source modified: `2026-06-25T17:02:09+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_c81dbe7a-7ff`
- session\_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

## Transcript

### 1. user (2026-06-25T17:01:08.863Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [MEDIUM] Sweep deletes ACCOUNT-GLOBAL Gemini files (>1h, any video/\*) ? blast radius under a shared GEMINI\_API\_KEY  
Location: `backend/providers/gemini.py:61-79,116`  
Detail: Uploads are created with `files.upload(file=video_path)` (`gemini.py:116`) with NO `display_name` or instance tag, so there is no way to scope the sweep to THIS app's own orphans. `cleanup_orphaned_files` therefore deletes EVERY file in the account whose `mime_type` starts with 'video/' and whose `create_time` is older than `grace_sec` (`gemini.py:67-78`). The Gemini File API is account-global per API key. If one GEMINI\_API\_KEY is shared across deployments (the alpha server + the Cloud Run worker + a dev box + an owner laptop all can share a key), then ANY one of them restarting will sweep every other instance's legitimately in-flight-but->1h video ? e.g. a long batch on another box, or a re-ingest mid-run. The 1h grace mitigates the common single-clip case (a clip's full lifetime is well under the 20-min processing cap + generation), but does not mitigate a genuinely concurrent long-running job under the same key. This is silent cross-instance data deletion triggered by an unrelated process's boot. FIX: either scope the sweep to this app's own uploads (tag uploads via `display_name` with an app/instance prefix on upload, then only delete files matching that prefix), or document the shared-key hazard loudly and gate the sweep behind an explicit opt-in config flag (default off) rather than running automatically whenever `ai_provider==gemini`.  
Suggested fix: Tag uploads with an app-scoped `display_name` and only sweep files matching that prefix, OR put the sweep behind an explicit opt-in flag + document that it is account-global deletion (unsafe to enable when the API key is shared).

Return `isReal` + `adjustedSeverity` (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

### 2. assistant (2026-06-25T17:01:11.240Z)

I'll verify this finding by examining the actual code in the PR.

### 3. user (2026-06-25T17:01:12.928Z)

f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep  
86705c1 chore: ruff format `gemini_refine.py` + `trajectory_hybrid.py` (#379)  
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)  
71a8241 chore(repo): fix corrupted .gitignore (UTF-16 tail) + ignore artifacts/, track product overview (#377)  
f7fb36e docs(audit): fix two accuracy nits ? D2 DoD wording + main.py line count (#376)  
---BRANCH---  
o1-p11/gemini-remote-file-gc

#### 4. user (2026-06-25T17:01:13.659Z)

```
backend/main.py | 25 ++++++++
backend/providers/gemini.py | 43 ++++++++
tests/test_provider_gemini.py | 104 ++++++++
3 files changed, 172 insertions(+)
```

#### 5. assistant (2026-06-25T17:01:16.351Z)

Let me read the actual changed code.

#### 6. user (2026-06-25T17:01:17.445Z)

```
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
import logging
import os
import time
+from datetime import datetime, timedelta, timezone
+from typing import Tuple

+from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+            live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
```

```

+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+         if deleted:
+             logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+         return deleted
+
+     def analyze_video(self, video_path: str, prompt: str,
                        chunk_duration: float = 0.0,
                        label: str = "") -> Tuple[list, dict]:

```

## 7. assistant (2026-06-25T17:01:19.840Z)

Now let me see how this is wired into main.py and read the full gemini.py context including the upload code.

## 8. user (2026-06-25T17:01:20.975Z)

```

diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure # standardized error/result contract
from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
    sys.exit(1)

+def _maybe_sweep_gemini_orphans(config) -> None:
+    """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+def main():
+    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
+    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
    if swept:
        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)

```

```

+
# CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
if args.video_path and not args.server and not args.app:
    print(f"[CLI] Processing video file: {args.video_path}")

```

## 9. user (2026-06-25T17:01:21.818Z)

```

85
86     def analyze_video(self, video_path: str, prompt: str,
87                       chunk_duration: float = 0.0,
88                       label: str = "") -> Tuple[list, dict]:
89         log_prefix = f"Gemini {label}" if label else "Gemini"
90
91         metrics: dict = {
92             "upload_time": 0.0,
93             "process_time": 0.0,
94             "gen_time": 0.0,
95         }
96
97         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
98         try:
99             size_mb = os.path.getsize(video_path) / (1024 * 1024)
100         except OSError:
101             size_mb = 0.0
102         if size_mb > MAX_UPLOAD_MB:
103             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
104                         f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
105                         f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
106             metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
107             metrics["oversize_skipped"] = True
108             return [], metrics
109
110         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111         logger.info(f"[{log_prefix}] Uploading {video_path}...")
112         t_upload_start = time.time()
113         video_file = None
114         for attempt in range(3):
115             try:
116                 video_file = self.client.files.upload(file=video_path)
117                 metrics["upload_time"] = time.time() - t_upload_start
118                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119                 break
120             except Exception as e:
121                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122                 if attempt < 2:
123                     time.sleep(2 * (attempt + 1))
124         if video_file is None:
125             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126             metrics["error"] = "upload failed after 3 attempts"
127             return [], metrics
128
129         # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130         # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131         t_process_start = time.time()
132         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap

```

```

133
134         def _processed():
135             nonlocal video_file
136             video_file = self.client.files.get(name=video_file.name)
137             return None if video_file.state.name == "PROCESSING" else video_file
138
139         try:
140             video_file = poll_until(_processed, process_policy,
141                                     label=f"{log_prefix} processing", logger=logger)
142         except PolicyTimeout:
143             logger.error(f"[{log_prefix}] Processing exceeded
144 {process_policy.max_wait_sec:.0f}s ? giving up.")
145             self._safe_delete(video_file)

```

#### 10. assistant (2026-06-25T17:01:25.423Z)

The finding's claims about the code are accurate. Let me confirm the upload has no `display_name` (already confirmed at line 116: `self.client.files.upload(file=video_path)`), and check whether there's any config flag gating this or any documentation of the shared-key hazard. Let me also check the test file and whether there's any existing scoping/config awareness.`

#### 11. user (2026-06-25T17:01:27.664Z)

```

backend\main.py:411:         "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
backend\main.py:1188: def _maybe_sweep_gemini_orphans(config) -> None:
backend\main.py:1189:     """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a
previous crashed run.
backend\main.py:1196:         api_key = os.environ.get("GEMINI_API_KEY")
backend\main.py:1200:         deleted = provider.cleanup_orphaned_files()
backend\main.py:1202:         logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote
file(s).", deleted)
backend\main.py:1204:         logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal):
%s", e)
backend\main.py:1342:     # 01/P11: sweep Gemini File-API uploads orphaned by a hard process
death (the per-call
backend\main.py:1345:     _maybe_sweep_gemini_orphans(global_config)
backend\main.py:1454:     # Resume self-scheduled work orphaned by the restart
(EXECUTION_POLICY.md P3): the
backend\providers\gemini.py:44:     def cleanup_orphaned_files(self, grace_sec: int = 3600) ->
int:
backend\providers\gemini.py:45:         """Sweep and delete video files orphaned on the Gemini
File API by a previous
backend\providers\gemini.py:63:             logger.warning("[Gemini GC] failed to list remote
files (skipping sweep): %s", e)
backend\providers\gemini.py:81:             logger.debug("[Gemini GC] skipping a file during
sweep: %s", e)
backend\providers\gemini.py:83:             logger.info("[Gemini GC] deleted %d orphaned video
file(s) older than %ds.", deleted, grace_sec)
backend\reporting\spec.yaml:35:     CREDENTIAL, not 'the video has no rallies'. Fix: put
GEMINI_API_KEY=<key> in a
backend\utils\app_home.py:62:     """`<app-home>/.env` ? the dotenv file the server prefers
(e.g. `GEMINI_API_KEY`)."
backend\storage\gdrive_api.py:131:         # ORPHAN the prior file (and reads would keep
returning the stale first match). Delete an
backend\eval\gemini_refine.py:17: Run (GEMINI_API_KEY in env):
backend\eval\gemini_refine.py:138:     api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:140:     raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:230:     api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:232:     raise SystemExit("GEMINI_API_KEY not set in env")
backend\ui\assets\index-ukuS9wxa.js:48: [Omitted long matching line]
backend\ui\assets\index-ukuS9wxa.js:60: [Omitted long matching line]

```

#### 12. user (2026-06-25T17:01:28.402Z)

```

backend\providers\gemini.py:46:         crashed run (a hard process death between
`files.upload` and `_safe_delete`
backend\providers\gemini.py:116:         video_file =
self.client.files.upload(file=video_path)
tests\test_provider_gemini.py:18:         client_mock.files.upload.return_value = file_mock
tests\test_provider_gemini.py:40:         client_mock.files.upload.return_value = file_mock
tests\test_provider_gemini.py:66:         client_mock.files.upload.return_value = file_mock
tests\test_provider_gemini.py:88:         client_mock.files.upload.assert_not_called()
tests\test_provider_gemini.py:99:         client_mock.files.upload.side_effect =
Exception("Upload err")

```

### 13. assistant (2026-06-25T17:01:35.136Z)

The finding's technical claims are all verified against the code:

1. **\*\*Upload has no `display\_name`/tag\*\*** ? confirmed at `gemini.py:116`:  
`self.client.files.upload(file=video\_path)` with no scoping.
2. **\*\*Sweep deletes any video file > grace\*\*** ? confirmed at `gemini.py:67-78`: it filters only on  
`mime\_type.startswith("video/")` and `created < cutoff`, with no app/instance scoping.
3. **\*\*No opt-in flag\*\*** ? confirmed in `main.py:1188-1204`: `\_maybe\_sweep\_gemini\_orphans` runs  
automatically whenever `ai\_provider == "gemini"` and `GEMINI\_API\_KEY` is present. There is no  
config flag gating it.
4. **\*\*Gemini File API is account-global per key\*\*** ? this is correct; the File API namespace is  
scoped to the API key, not to an app instance.

Now let me assess the real-world blast radius for *this* project to calibrate severity. The key  
questions: (a) is key-sharing across instances actually plausible here, and (b) does the 1h  
grace + 20min processing cap meaningfully bound the risk.

### 14. user (2026-06-25T17:01:37.025Z)

```

144         self._safe_delete(video_file)
145         metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
146         return [], metrics
147     except Exception as e:
148         logger.error(f"[{log_prefix}] Status check failed: {e}")
149         self._safe_delete(video_file)
150         metrics["error"] = f"status check failed: {e}"
151         return [], metrics
152     if video_file.state.name == "FAILED":
153         logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
154         self._safe_delete(video_file)
155         metrics["error"] = f"video processing failed: {video_file.error.message}"
156         return [], metrics
157
158     metrics["process_time"] = time.time() - t_process_start
159     logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
160
161     # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
162     # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
163     # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
164     logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
165     t_gen_start = time.time()
166     gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167     raw_text = None
168
169     def _generate():

```

```

170         return self.client.models.generate_content(
171             model=self.model_name,
172             contents=[video_file, prompt],
173             config=types.GenerateContentConfig(
174                 response_mime_type="application/json", temperature=0.0),
175         )
176
177     try:
178         response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179         raw_text = response.text
180         metrics["gen_time"] = time.time() - t_gen_start
181         logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182     except Exception as e:                                # PolicyTimeout (hang) or the last 503 ?
both end here
183         logger.error(f"[{log_prefix}] generate gave up after retries: {e}")

```

### 15. user (2026-06-25T17:01:37.538Z)

```

100
101         provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
102         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
103         assert len(segments) == 0
104
105
106
107     # --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
111         f.name = name
112         f.mime_type = mime
113         f.create_time = create_time_iso
114         f.state = MagicMock()
115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
files."""
121         from datetime import datetime, timedelta, timezone
122         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
123         fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
124         files = [
125             _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
126             _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
127             _file_mock("old-img", "image/png", stale),          # non-video -> kept
128             _file_mock("notts", "video/mp4", None),            # no timestamp -> kept (safe)
129         ]
130         with patch("backend.providers.gemini.genai"):
131             provider = GeminiProvider(api_key="k", model_name="m")
132             provider._client = MagicMock()
133             provider._client.files.list.return_value = iter(files)
134             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
135             assert deleted == 1
136             provider._client.files.delete.assert_called_once_with(name="old-vid")
137
138
139     def test_cleanup_orphaned_files_list_failure_is_swallowed():
140         """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
141         with patch("backend.providers.gemini.genai"):

```

```

142         provider = GeminiProvider(api_key="k", model_name="m")
143         provider._client = MagicMock()
144         provider._client.files.list.side_effect = RuntimeError("network")
145         assert provider.cleanup_orphaned_files() == 0
146
147
148     def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149         """A delete() failure on one file must not abort the sweep."""
150         from datetime import datetime, timedelta, timezone
151         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152         files = [
153             _file_mock("a", "video/mp4", stale),
154             _file_mock("b", "video/mp4", stale),
155         ]
156         with patch("backend.providers.gemini.genai"):
157             provider = GeminiProvider(api_key="k", model_name="m")
158             provider._client = MagicMock()
159             provider._client.files.list.return_value = iter(files)
160             # First delete raises, second succeeds ? _safe_delete swallows it.
161             provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163             assert deleted == 1 # only the second file counted
164
165
166
167     # --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == [] # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()
186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
197 startup)."""
198         import backend.main as main_mod
199         cfg = MagicMock()
200         cfg.ai_provider = "gemini"
201         cfg.ai_model = "gemini-2.5-flash"
202         provider = MagicMock()
203         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
204         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
205         monkeypatch.setenv("GEMINI_API_KEY", "k")
206         # Must not raise.

```

```
206         main_mod._maybe_sweep_gemini_orphans(cfg)
207         provider.cleanup_orphaned_files.assert_called_once()
208
```

## 16. assistant (2026-06-25T17:01:52.473Z)

I have a complete picture. Let me assess severity carefully and adversarially.

The finding's factual claims are all correct. The real question is the severity calibration. Key considerations:

**\*\*Supporting the finding (real risk):\*\***

- The sweep is genuinely account-global and indiscriminate within `video/\*` ? no instance tagging, no opt-in flag.
- The project's own memory explicitly documents shared-key, multi-instance topology: "the alpha server + the Cloud Run worker + a dev box + an owner laptop all can share a key" is consistent with `[ui-driven-cloud-serving-gap]` (alpha server `api.khelsutra.guru`), `[cloudrun-gpu-serving-gotchas]` (Cloud Run L4 GPU worker), and a local dev box ? all of which use `GEMINI\_API\_KEY` from env. So the shared-key scenario is not hypothetical for this project.
- A concurrent long-running job (large match, batch, re-ingest) on instance B that exceeds 1h wall-clock while instance A boots ? instance A deletes B's in-flight file ? B's `generate\_content` fails. This is silent, cross-instance, triggered by an unrelated boot.

**\*\*Mitigating (why not HIGH):\*\***

- The single-clip lifetime is genuinely bounded: 20-min processing cap + ~3-min generate. The 1h grace comfortably covers any \*single\* `analyze\_video` call, so the common case is safe.
- The blast radius is data that is \*re-creatable\* (a re-upload), not durable user data ? the file is an ephemeral processing artifact. A deleted in-flight file causes a failed/retried job, not permanent data loss. (Note: the upload retry loop is only 3 attempts and won't recover from a mid-generate deletion, so the active job does fail.)
- Triggering requires a genuinely >1h concurrent job under a shared key during a boot ? a real but narrow window, not the default path.

This is a real, correctly-identified design hazard with a plausible (project-documented) trigger, but the impact is a recoverable failed job rather than durable data loss, and the trigger window is narrow. That lands it solidly at MEDIUM ? not downgradeable to LOW because the project's own architecture (documented shared-key multi-instance serving) makes the precondition realistic, and the fix (display\_name tagging or an opt-in flag) is cheap and standard. The suggested fixes are both sound and idiomatic.

## 17. user (2026-06-25T17:02:01.175Z)

Structured output provided successfully

## 18. assistant (2026-06-25T17:02:09.582Z)

I've completed the adversarial verification of this finding.

**\*\*Verdict: REAL, MEDIUM\*\*** (severity confirmed, not adjusted).

The finding's technical claims all check out against the actual code on branch `o1-p11/gemini-remote-file-gc`:

- `backend/providers/gemini.py:116` ? `files.upload(file=video\_path)` has no `display\_name`/instance tag, so orphans can't be scoped to this app.
- `backend/providers/gemini.py:44-83` ? `cleanup\_orphaned\_files` deletes any account file matching `mime\_type.startswith("video/")` AND older than `grace\_sec`, with no app scoping.
- `backend/main.py:1188-1204` + `:1345` ? the sweep runs automatically on every startup when `ai\_provider=="gemini"` and `GEMINI\_API\_KEY` is set; there is no opt-in flag.

The shared-key/multi-instance precondition is not hypothetical for this project ? the repo's own memory documents exactly that topology (alpha server `api.khelsutra.guru`, Cloud Run L4 GPU worker, dev box, owner laptop, all using `GEMINI\_API\_KEY` from env). So an unrelated instance booting can silently delete another instance's >1h in-flight upload.

It stays at MEDIUM rather than escalating to HIGH because the per-clip lifetime is bounded by the 20-min processing cap + ~3-min generate (the 1h grace fully covers any single ``analyze_video`` call), the deleted artifact is an ephemeral re-creatable upload rather than durable data, and the trigger window (a genuinely >1h concurrent job under a shared key coinciding with a boot) is narrow. Both suggested fixes (app-prefixed ``display_name`` scoping, or a default-off opt-in flag) are standard and correct.